CS 301
High-Performance Computing

Assignment 03 - Serial Interpolation on Scattered Data

Dhruvi Jobanputra (202301401)
Prasham Shah (202301252)

February 18, 2026

Contents

1 Introduction **3**

2 Problem Description **3**

3 Implementation Approach **3**
3.1 Data Layout and Grid Indexing 3
3.2 Scatter-to-Grid Mapping 4
3.3 Interpolation Workflow 4

4 Results **5**

5 Analysis Tasks **7**

6 Code Optimization and Performance Improvement **9**
6.1 Local Speedup 9
6.2 HPC Speedup 9

7 Conclusion **11**

1 Introduction

Interpolation of scattered data onto a structured grid is a common operation in scientific computing and numerical simulations. Given randomly distributed spatial samples, the goal is to reconstruct values on a regular mesh suitable for visualization or further computation.

In this assignment, bilinear interpolation is implemented to map scattered 2D points onto a structured grid. The performance of the serial algorithm is evaluated on both a local workstation and an HPC cluster across multiple problem sizes.

2 Problem Description

Given a set of scattered points (x_i, y_i) in a unit square domain, the task is to interpolate their values onto a structured grid of size $N_X \times N_Y$. Each scattered point contributes to the four nearest grid nodes based on bilinear interpolation weights determined by its relative position within the grid cell.

The interpolation is repeated for multiple iterations using different particle datasets. Execution time is measured for varying grid resolutions and particle counts.

3 Implementation Approach

To efficiently process the scattered points and map them to the structured grid, careful consideration must be given to data layout, grid indexing, and the interpolation workflow.

3.1 Data Layout and Grid Indexing

The structured grid is fundamentally 2D, but to ensure contiguous memory access and improve cache utilization, it is stored as a one-dimensional array in row-major order.

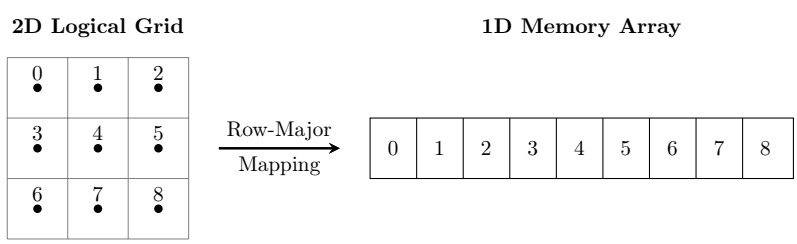

Figure 1: Mapping a 2D structured grid to a 1D array in row-major order.

A grid node at index (i, j) is accessed sequentially using the formula:

$$mesh[j \cdot GRID_X + i]$$

This indexing strategy allows the algorithm to locate the bottom-left corner of any grid cell in $O(1)$ time.

3.2 Scatter-to-Grid Mapping

Each scattered particle located inside a grid cell distributes its value to the four neighboring grid nodes. Local coordinates inside the cell are computed as:

$$l_x = x - X_i$$
$$l_y = y - Y_j$$

The bilinear interpolation weights, representing the proportional areas of the opposing sub-rectangles, are calculated as follows:

$$w_{00} = (\Delta x - l_x)(\Delta y - l_y)$$
$$w_{10} = l_x(\Delta y - l_y)$$
$$w_{01} = (\Delta x - l_x)l_y$$
$$w_{11} = l_x l_y$$

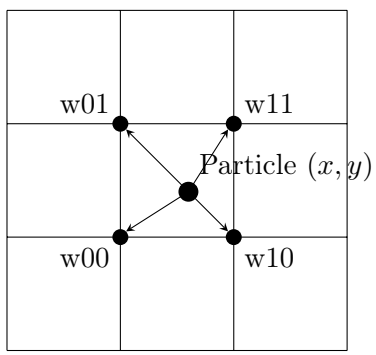

Figure 2: Particle mapping inside a grid cell and its bilinear weight distribution to four neighboring nodes.

3.3 Interpolation Workflow

The algorithm executes a constant-time workflow for every particle independently. Because the operations per particle do not depend on the overall grid size, the total time complexity scales linearly with the number of scattered points.

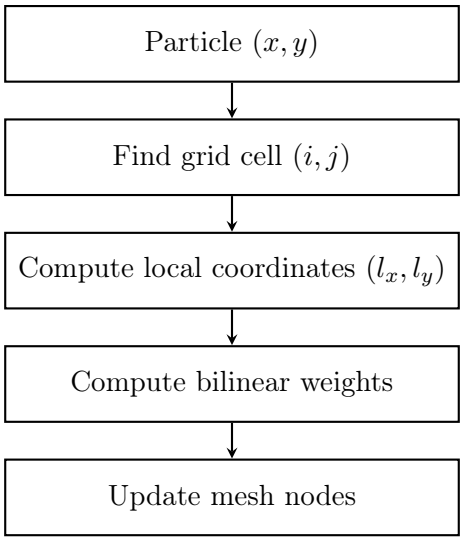

Figure 3: Sequential interpolation workflow per scattered particle.

4 Results

Execution time was measured for five configurations on both systems.

Case	Points	Local (s)	HPC (s)
a	0.9M	0.1039	0.37
b	5M	0.5586	1.55
c	3.6M	0.4180	1.17
d	20M	2.3163	6.46
e	14M	1.9516	4.39

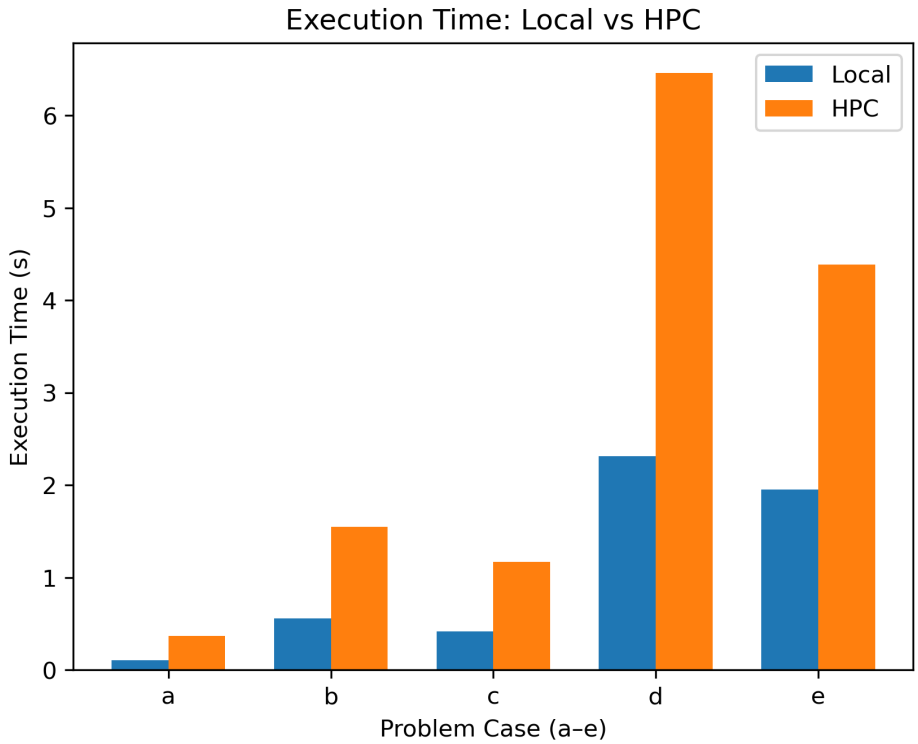

Figure 4: Execution time comparison of serial interpolation across problem sizes on local and HPC systems.

5 Analysis Tasks

Q1. Compute the theoretical time complexity.

In the particle–grid interpolation method, the algorithm iterates over all scattered particles. Each particle requires a fixed number of operations such as cell identification, weight evaluation, and four grid updates. Since the work per particle is constant, the computational cost grows linearly with the number of particles:

$$T(N) = O(N)$$

If the interpolation process is repeated for `Maxiter` iterations, the total complexity becomes:

$$T(N) = O(N \cdot \texttt{Maxiter})$$

Thus, the algorithm scales proportionally with both particle count and iteration count.

Q2. Estimate arithmetic operations per particle.

For each particle (x_i, y_i, f_i), the following arithmetic operations are performed:

- Cell index computation: 2 divisions

$$i = \frac{x_i}{\Delta x}, \quad j = \frac{y_i}{\Delta y}$$

- Grid coordinate reconstruction: 2 multiplications

$$X_i = i\Delta x, \quad Y_j = j\Delta y$$

- Local offset inside cell: 2 subtractions

$$l_x = x_i - X_i, \quad l_y = y_i - Y_j$$

- Bilinear weight evaluation: 8 subtractions + 4 multiplications
- Grid accumulation: 4 multiplications + 4 additions

$$F_{p,q} += w_{p,q}\, f_i$$

The total arithmetic workload is approximately 24–26 floating-point operations per particle.

Q3. Perform memory access analysis and analyze cache behavior.

During interpolation of one particle, the following memory accesses occur:

- Three reads from particle data: (x_i, y_i, f_i)
- Four read–modify–write updates to grid nodes:

$$F_{i,j},\ F_{i+1,j},\ F_{i,j+1},\ F_{i+1,j+1}$$

Cache characteristics:

- The scattered particle array is accessed sequentially, giving good spatial locality.
- Grid updates occur at positions determined by particle location, producing irregular access.
- When several nearby particles map to the same cell, temporal locality improves.
- For large meshes exceeding cache capacity, capacity misses increase significantly.

Therefore, interpolation performance is largely limited by memory access rather than arithmetic cost.

Q4. Suggest further optimizations under better hardware assumptions.
With more advanced hardware support, several optimizations can improve efficiency:

- **Data layout optimization:** Using a Structure-of-Arrays (SoA) layout instead of Array-of-Structures (AoS) improves cache utilization and enables vector operations.
- **SIMD vectorization:** Multiple particles can be processed simultaneously using vector instructions.
- **Parallel execution:** Particle loops can be distributed across cores or nodes using OpenMP or MPI.

These optimizations primarily target memory throughput and parallel scalability.

Q5. Measure and compare execution time and explain observed differences.
The execution time for different configurations is shown in Fig. **??**. Runtime increases approximately linearly with problem size on both systems. However, the HPC cluster exhibits higher execution time than the lab PC for this serial implementation.
This behavior can be attributed to:

- Lower single-core clock frequency on HPC nodes.
- Higher memory latency due to shared memory hierarchy.
- Lack of parallelization in the serial algorithm.
- Memory-bound behavior caused by irregular grid updates and cache misses.

Thus, although HPC systems provide greater parallel capability, the serial interpolation code cannot benefit from it.

Q6. Explain implementation approach pictorially.
The interpolation workflow and grid mapping are illustrated in the diagram presented in Section 3. The figure shows how each scattered particle located inside a grid cell contributes to its four neighboring structured grid nodes through bilinear weights. This pictorial representation clarifies the data layout, indexing scheme, and scatter-to-grid update process independent of the code implementation.

6 Code Optimization and Performance Improvement

The baseline interpolation implementation was optimized by eliminating floating-point divisions and reducing repeated memory indexing. Grid spacing inverses were precomputed to replace divisions with multiplications, and mesh index computation was simplified by computing the base grid index once per particle. These changes reduce arithmetic cost while preserving correctness.

Performance was evaluated on both local and HPC systems.

6.1 Local Speedup

Case	Baseline (s)	Optimized (s)	Speedup
a	0.1039	0.0816	1.27×
b	0.5586	0.4455	1.25×
c	0.4180	0.3228	1.30×
d	2.3163	1.7745	1.31×
e	1.9516	1.9433	1.00×

Table 1: Optimization speedup on local system

6.2 HPC Speedup

Case	Baseline (s)	Optimized (s)	Speedup
a	0.37	0.28	1.32×
b	1.55	1.17	1.32×
c	1.17	0.99	1.18×
d	6.46	5.07	1.27×
e	4.39	4.53	0.97×

Table 2: Optimization speedup on HPC system

The optimization achieves approximately 25–30% speedup for most configurations on both systems by reducing arithmetic overhead. For the largest grid configuration (case e), performance improvement is limited because the algorithm becomes memory-bound, and memory access latency dominates runtime.

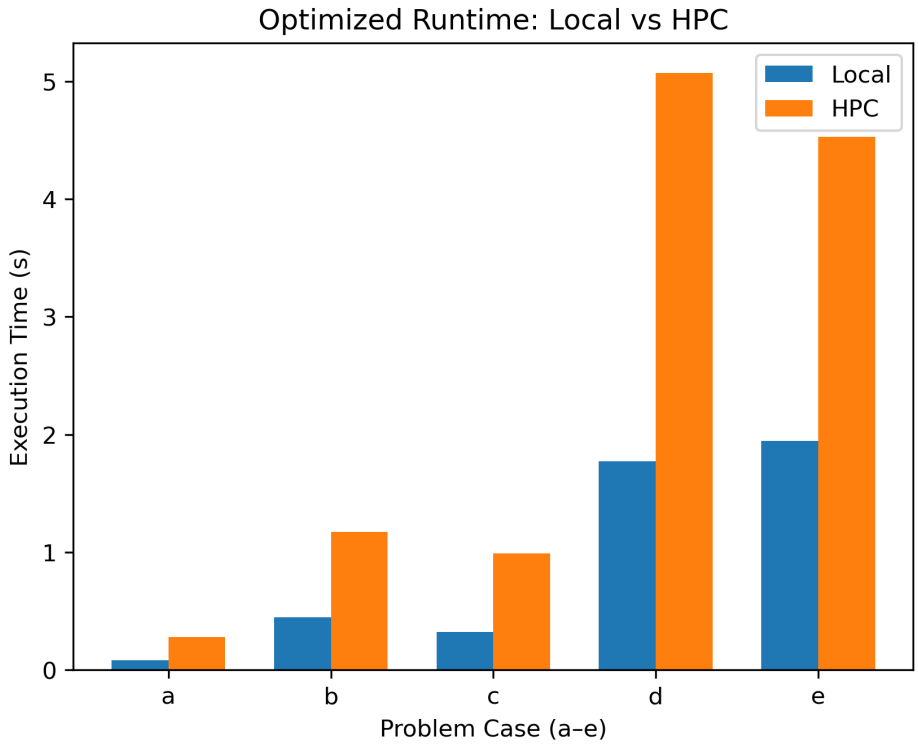

Figure 5: Optimized interpolation runtime comparison between Local and HPC systems across problem sizes.

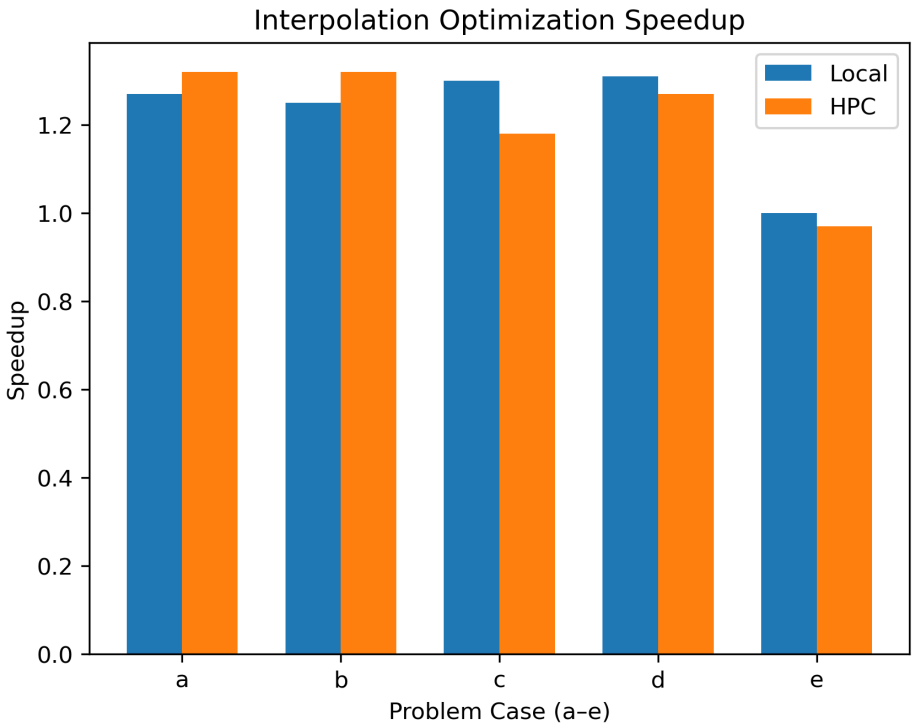

Figure 6: Speedup achieved by interpolation optimization on Local and HPC systems.

7 Conclusion

- The serial bilinear interpolation algorithm for mapping scattered particle data onto a structured grid was successfully implemented and validated using the reference dataset.
- The theoretical time complexity $O(N \cdot \texttt{Maxiter})$ was confirmed experimentally through near-linear scaling of runtime with particle count.
- Memory access analysis showed that the algorithm is predominantly memory-bound due to irregular grid read–modify–write operations and limited cache reuse.
- Performance comparison indicated that the lab PC outperformed the HPC cluster for the serial implementation because of higher single-core frequency and lower memory latency.
- An optimized interpolation version reduced arithmetic and indexing overhead while preserving correctness, achieving consistent runtime improvement across all configurations.
- The results demonstrate that memory access patterns and cache behavior dominate interpolation performance on modern architectures.
- Overall, the study highlights that even simple low-level optimizations can significantly improve serial execution efficiency for memory-bound algorithms.